

Module 4: Apriori & FP–Growth Pattern Mining

Support/Confidence, Apriori Join & Prune, FP–Tree Mining & Lift

Module IV: Frequent Pattern Mining & Association Analysis — 7 Topics Syllabus

- | | |
|---|--|
| 1. Basic Concepts of Frequent Pattern Mining | 2. Association Rules (Support & Confidence) |
| 3. Correlation Analysis (Lift, Chi-Square, Kulczynski) | 4. Frequent Itemset Mining Methods Landscape (Apriori, FP–Growth, ECLAT) |
| 5. The Apriori Algorithm (Formal Proof, Join & Prune Steps) | 6. Pattern–Growth Approach (FP–Growth, FP–Tree, Conditional Bases) |
| 7. Interesting Pattern Evaluation Methods (Null–Invariance) | |

Topics 1 – 2: Market Basket Analysis & Association Rules

Frequent Patterns are itemsets, subsequences, or substructures that appear frequently in a transactional dataset (with frequency not less than a user-specified threshold).

Market Basket Analysis investigates customer buying habits by finding associations between the different items that customers place in their shopping baskets:

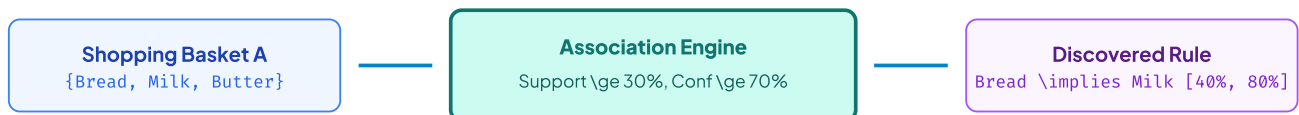


FIGURE 4.1: MARKET BASKET ASSOCIATION RULE EXTRACTION

Mathematical Formulations:

- Itemset:** A set of items $X = \{i_1, i_2, \dots, i_k\}$. A k -itemset contains exactly k items.
- Support Count ($\sigma(X)$):** The absolute number of transactions in database D containing itemset X :

$$\sigma(X) = |\{T \in D \mid X \subseteq T\}|$$

- Relative Support (s):** Fraction of transactions containing X :

$$\text{Support}(X) = \frac{\sigma(X)}{|D|}$$

- Association Rule:** An expression $A \implies B$ where $A \subset I, B \subset I$, and $A \cap B = \emptyset$.

$$\text{Support}(A \implies B) = P(A \cup B) = \frac{\sigma(A \cup B)}{|D|}$$

$$\text{Confidence}(A \implies B) = P(B \mid A) = \frac{\sigma(A \cup B)}{\sigma(A)} = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

Topic 5: The Apriori Algorithm (Level-Wise Search)

The Apriori Downward Closure Property (Apriori Property)

Theorem: All non-empty subsets of a frequent itemset must also be frequent.

Formal Proof: Let I be an itemset and $I' \subseteq I$. Every transaction $T \in D$ that contains I must inherently contain all of its subsets I' (since $I \subseteq T \implies I' \subseteq T$). Therefore:

$$\sigma(I') = |\{T \in D \mid I' \subseteq T\}| \geq |\{T \in D \mid I \subseteq T\}| = \sigma(I)$$

If I is frequent ($\sigma(I) \geq \text{min_sup}$), then $\sigma(I') \geq \text{min_sup}$, proving that I' is also frequent.

Pruning Rule (Contrapositive): If any $(k-1)$ -subset of a candidate k -itemset C_k is infrequent ($< \text{min_sup}$), then C_k cannot be frequent and is immediately pruned before database scanning!

Two-Step Candidate Generation Mechanism:

1. **Join Step** ($L_{k-1} \bowtie L_{k-1} \rightarrow C_k$): Join two frequent $(k-1)$ -itemsets $l_1, l_2 \in L_{k-1}$ if their first $k-2$ items are identical and the last item of l_1 is lexicographically smaller than l_2 :

$$(l_1[1] = l_2[1]) \wedge \dots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$$

2. **Prune Step:** For each candidate $c \in C_k$, check all $(k-1)$ -subsets of c . If any subset $\notin L_{k-1}$, delete c from C_k .

Step-by-Step Complete Apriori Trace on 9 Transactions

Transaction Database (D):

TID	Items Purchased
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Parameter: $\text{min_sup} = 2$ ($\approx 22.2\%$), $\text{min_conf} = 70\%$.

Complete Iterative Trace:

1. Pass 1:

- $C_1 = \{\{I1\} : 6, \{I2\} : 7, \{I3\} : 6, \{I4\} : 2, \{I5\} : 2\}$.
- All items have count $\geq 2 \implies L_1 = C_1$.

2. Pass 2:

- Join $L_1 \bowtie L_1 \rightarrow C_2 = \{\{I1, I2\}, \{I1, I3\}, \{I1, I4\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}, \{I3, I4\}, \{I3, I5\}, \{I4, I5\}\}$.
- Scan DB $\rightarrow L_2 = \{\{I1, I2\} : 4, \{I1, I3\} : 4, \{I1, I5\} : 2, \{I2, I3\} : 4, \{I2, I4\} : 2, \{I2, I5\} : 2\}$.
- Infrequent items dropped: $\{I3, I4\} : 0, \{I3, I5\} : 1, \{I4, I5\} : 0$.

3. Pass 3:

- Join $L_2 \bowtie L_2 \rightarrow$ Candidates:
 - $\{I1, I2\} \bowtie \{I1, I3\} \rightarrow \{I1, I2, I3\}$. Subsets: $\{I1, I2\}, \{I1, I3\}, \{I2, I3\} \in L_2 \implies$ **Keep**.
 - $\{I1, I2\} \bowtie \{I1, I5\} \rightarrow \{I1, I2, I5\}$. Subsets: $\{I1, I2\}, \{I1, I5\}, \{I2, I5\} \in L_2 \implies$ **Keep**.
 - $\{I2, I3\} \bowtie \{I2, I4\} \rightarrow \{I2, I3, I4\}$. Subset $\{I3, I4\} \notin L_2 \implies$ **Pruned!**
- $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$.
- Scan DB $\rightarrow L_3 = \{\{I1, I2, I3\} : 2, \{I1, I2, I5\} : 2\}$.

4. **Pass 4:** Join $L_3 \bowtie L_3 \rightarrow \{I1, I2, I3, I5\}$. Subset $\{I2, I3, I5\} \notin L_3 \implies$ **Pruned!** Algorithm terminates.

Rule Generation from Frequent Itemset $\{I1, I2, I5\}$ ($\sigma = 2$):

- Rule $\{I1, I5\} \implies \{I2\}$: $\text{Conf} = \frac{\sigma(I1, I2, I5)}{\sigma(I1, I5)} = \frac{2}{2} = 100\% \geq 70\% \implies$ **StrongRule!**
- Rule $\{I2, I5\} \implies \{I1\}$: $\text{Conf} = \frac{\sigma(I1, I2, I5)}{\sigma(I2, I5)} = \frac{2}{2} = 100\% \geq 70\% \implies$ **StrongRule!**
- Rule $\{I5\} \implies \{I1, I2\}$: $\text{Conf} = \frac{\sigma(I1, I2, I5)}{\sigma(I5)} = \frac{2}{2} = 100\% \geq 70\% \implies$ **StrongRule!**

- Rule $\{I1, I2\} \Rightarrow \{I5\}$: $\text{Conf} = \frac{\sigma(I1, I2, I5)}{\sigma(I1, I2)} = \frac{2}{4} = 50\% < 70\% \Rightarrow \text{Rejected!}$

Topic 4: Vertical Data Format (ECLAT Algorithm)

ECLAT (Equivalence Class Transformation) mines frequent itemsets using a **Vertical Data Format** where each item is associated with its **TID-list** (list of transaction IDs containing the item):

Item	TID-List	Support Count
I1	{T100, T400, T500, T700, T800, T900}	6
I2	{T100, T200, T300, T400, T600, T800, T900}	7
I3	{T300, T500, T600, T700, T800, T900}	6
I4	{T200, T400}	2
I5	{T100, T800}	2

TID-List Intersections for 2-Itemsets:

- $\text{TID}(I1, I2) = \text{TID}(I1) \cap \text{TID}(I2) = \{T100, T400, T800, T900\} \implies \text{Count} = 4 \geq 2$
- $\text{TID}(I1, I3) = \{T500, T700, T800, T900\} \implies \text{Count} = 4 \geq 2$
- $\text{TID}(I1, I5) = \{T100, T800\} \implies \text{Count} = 2 \geq 2$
- $\text{TID}(I2, I3) = \{T300, T600, T800, T900\} \implies \text{Count} = 4 \geq 2$
- $\text{TID}(I2, I4) = \{T200, T400\} \implies \text{Count} = 2 \geq 2$
- $\text{TID}(I2, I5) = \{T100, T800\} \implies \text{Count} = 2 \geq 2$

Topic 6: The FP-Growth Algorithm (Mining Without Candidate Generation)

FP-Growth (Frequent Pattern Growth) overcomes the major performance bottlenecks of the Apriori algorithm (repeated full database scans and huge candidate itemset generation C_k).

The Two-Step FP-Growth Architecture:

1. Step 1: Construct the Compact FP-Tree:

- Scan database once to compute support counts of 1-itemsets; sort frequent items in descending order of support ($I2 : 7 > I1 : 6 > I3 : 6 > I4 : 2 > I5 : 2$).
- Scan database a second time. For each transaction, insert items into a shared prefix tree (FP-Tree). Shared transaction prefixes share branch nodes, incrementing branch counts.
- Construct the **Header Table** containing item heads and horizontal linked-list node pointers traversing all identical items in the tree.

2. Step 2: Mine the FP-Tree Recursively (Bottom-Up):

We extract Conditional Pattern Bases and construct Conditional FP-Trees starting from the bottom of the Header Table:

Item	Conditional Pattern Base	Conditional FP-Tree	Frequent Patterns Generated
I5	$\{I2, I1 : 1\}, \{I2, I1, I3 : 1\}$	$\langle I2 : 2, I1 : 2 \rangle$	$\{I2, I5 : 2\}, \{I1, I5 : 2\}, \{I2, I1, I5 : 2\}$
I4	$\{I2 : 1\}, \{I2, I1 : 1\}$	$\langle I2 : 2 \rangle$	$\{I2, I4 : 2\}$
I3	$\{I2, I1 : 2\}, \{I2 : 2\}, \{I1 : 2\}$	$\langle I2 : 4, I1 : 4 \rangle$	$\{I2, I3 : 4\}, \{I1, I3 : 4\}, \{I2, I1, I3 : 2\}$
I1	$\{I2 : 4\}$	$\langle I2 : 4 \rangle$	$\{I2, I1 : 4\}$

Topic 3 & 7: Interesting Pattern Evaluation Metrics

Lift Correlation Formulation:

$$\text{Lift}(A, B) = \frac{P(A \cup B)}{P(A)P(B)} = \frac{\text{Confidence}(A \implies B)}{\text{Support}(B)}$$

Metric Name	Mathematical Formula	Interpretation & Properties
Lift	$\text{Lift}(A, B) = \frac{P(A \cup B)}{P(A)P(B)}$	> 1 : Positively correlated. $= 1$: Independent. < 1 : Negatively correlated.
Max-Confidence	$\text{Max_Conf}(A, B) = \max(P(A \mid B), P(B \mid A))$	Null-invariant metric.
Kulczynski (Kulc)	$\text{Kulc}(A, B) = \frac{1}{2} \left(P(A \mid B) + P(B \mid A) \right)$	Arithmetic mean of conditional probabilities; Null-invariant (unaffected by total transaction size $ D $).
Imbalance Ratio (IR)	$\text{IR}(A, B) = \frac{ \text{supp}(A) - \text{supp}(B) }{\text{supp}(A) + \text{supp}(B) - \text{supp}(A \cup B)}$	Measures asymmetry in itemset support; used in tandem with Kulc.
Cosine (Itemsets)	$\text{Cosine}(A, B) = \frac{P(A \cup B)}{\sqrt{P(A)P(B)}} = \sqrt{P(A \mid B)P(B \mid A)}$	Geometric mean of conditional probabilities; Null-invariant.

Top BIT Mesra Exam Questions & Answers (Module IV)

Q1. Compare Apriori and FP-Growth algorithms across 5 critical engineering metrics. (8 Marks)

1. **Candidate Generation:** Apriori generates millions of candidate k -itemsets (C_k); FP-Growth generates zero candidates.
2. **Database Scans:** Apriori requires $k_{\max} + 1$ full scans of D ; FP-Growth requires exactly 2 database scans.
3. **Data Structure:** Apriori uses flat transactional arrays and hash trees; FP-Growth builds a highly compressed prefix tree with linked header tables.
4. **Memory Usage:** Apriori requires huge RAM for candidate sets C_k ; FP-Growth is memory compact via shared transaction prefixes.
5. **Mining Strategy:** Apriori is breadth-first (level-wise); FP-Growth is depth-first (divide-and-conquer).

Q2. Why is Support-Confidence framework insufficient for evaluating pattern interestingness? (6 Marks)

Answer: A rule can have high support and high confidence yet be completely uninteresting or deceptive. For example, in a grocery store where 75% of all customers purchase coffee, the rule $\text{Tea} \Rightarrow \text{Coffee}$ might have 75% confidence simply because coffee is popular overall, even though drinking tea might actually decrease the likelihood of buying coffee (negative correlation). Metrics like **Lift** and **Kulczynski** normalize for base individual item probabilities, exposing true causal affinity.

Q3. What is Null-Invariance in pattern evaluation and why is it crucial for massive transaction databases? (6 Marks)

Answer: A pattern interestingness measure M is **Null-Invariant** if its value is completely unaffected by the number of transactions that contain neither item A nor item B ($0 - 0$ negative transactions). In web transaction logs and massive e-commerce databases with millions of total transactions, the number of null transactions is astronomical. Non-null-invariant measures (like χ^2 and Lift) become severely distorted by total transaction count $|D|$, whereas null-invariant measures (like Kulczynski and Cosine) accurately measure true item affinity regardless of database size.